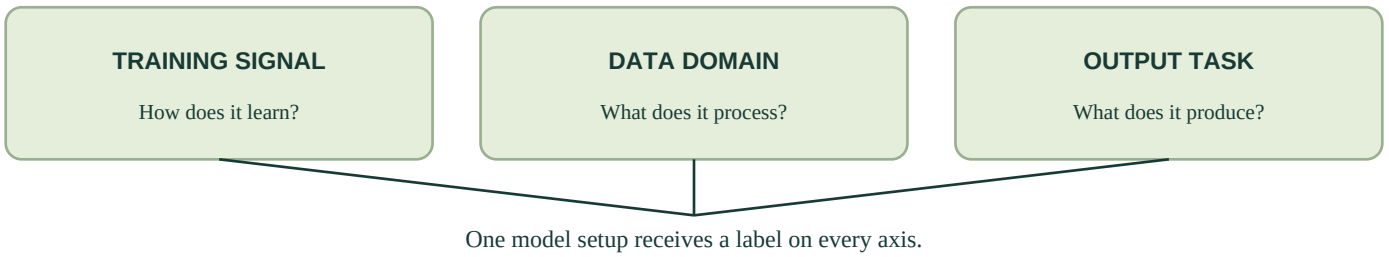
Three questions, one classification

Pavan Naga Phanendra Kancheti | AIML 501: Model Development

Purpose: compare ten algorithms and model families by training signal, data domain, and task. These are independent axes: a model can have several domain and task assignments. This guide covers selected methods, not every branch of machine learning.

Axis	Classification rule	Examples
Learning paradigm	Identify where the training targets come from.	Supervised: external labels. Unsupervised: structure without task labels. Self-supervised: targets derived from the data.
Data / application domain	Identify the input representation and application context.	Tabular records; computer vision; natural language processing (NLP).
Task	Identify the required output.	Regression, classification, clustering, dimensionality reduction, generation.



One model setup receives a label on every axis.

Why these categories?

Training signal explains how learning occurs; domain explains what data the method processes; task explains what the user wants. Treating these as separate axes prevents a task such as generation from being confused with a data domain. Generative AI is a task-oriented umbrella spanning text, images, audio, and other data.

How I classify each entry

1. State a specific training setup. 2. Check the source of targets. 3. Assign the output task. 4. Assign domains based on the input representation. 5. Record overlaps and exceptions instead of forcing one exclusive category. 6. Check the mechanism against documentation or original research [1]-[8].

Boundary example: a decision tree that classifies fruit from measured color and shape is supervised classification on tabular image features. A CNN trained on labeled pixels is supervised image classification. Both can address vision, but their input representations differ [1], [5].

Training, domain, and task

Model / method	Training setup shown	Primary task	Common input / domain
Linear regression	Supervised	Regression	Tabular numeric features
Decision tree	Supervised	Classification / regression	Tabular; extracted image/text features
Random forest	Supervised	Classification / regression	Tabular; extracted image/text features
Gradient boosting	Supervised	Classification / regression	Tabular; extracted image/text features
CNN classifier	Supervised in this setup	Classification	Images / computer vision
K-means	Unsupervised	Clustering	Numeric vectors; image/text embeddings
PCA	Unsupervised	Dimension reduction	Numeric vectors; image/text features
GPT-style transformer	Unsupervised umbrella: self-supervised pretraining	Text generation	Tokens / NLP
GAN	Unsupervised, unconditional setup	Generation	Images in the example
Diffusion model	Unsupervised umbrella: self-supervised denoising	Generation	Images in the example

Important classification boundaries

CNN and transformer name architectures, while supervised and self-supervised describe training. A CNN can also be used in a generative system; a transformer can be trained for classification. Architecture alone does not determine the learning paradigm [5], [6].

GPT pretraining predicts tokens using targets derived from text. Self-supervised learning is often grouped broadly with unsupervised learning, but the explicit label here makes the source of targets clear. Later training may use human-provided targets or feedback; the paradigm must be specified by stage [6].

An unconditional GAN requires no external semantic labels, even though the discriminator uses real/generated indicators. Diffusion denoising obtains targets by adding known noise to data. Conditional variants can use labels or paired data; they should be classified according to that specific setup [7], [8].

Learn from labeled examples

01 / Linear regression

Mechanism: Fits a weighted sum of features to predict a continuous target. **Example:** Estimate house prices from area, location, and age.
Selection and limitation: Useful interpretable baseline; nonlinear effects require suitable features. Evaluate with MAE or RMSE on held-out data. [1]

02 / Decision tree

Mechanism: Recursively splits feature space to improve label purity or reduce target error. **Example:** Classify fruit from measured color and shape.
Selection and limitation: Readable paths, but deep trees can overfit. Control depth and evaluate classification with precision, recall, or F1. [1]

03 / Random forest

Mechanism: Fits trees with bootstrap samples and randomized feature choices, then votes or averages.
Example: Predict customer churn from account and usage records.
Selection and limitation: Trees can train independently; averaging reduces variance. Interpretability is lower than a single tree. [2]

04 / Gradient boosting

Mechanism: Adds weak learners sequentially to reduce the current loss, commonly using negative gradients.
Example: Predict insurance claim costs from customer records.
Selection and limitation: Captures nonlinear patterns; learning rate, tree complexity, and number of stages affect generalization. [2]

05 / CNN classifier

Mechanism: Learns shared convolutional filters and spatial features, then maps them to class labels. **Example:** Identify defects in labeled product photographs.
Selection and limitation: Image structure is useful, but label quality and distribution shift matter. Evaluate on separate images, avoiding near-duplicate leakage. [5]

Find structure. Generate content.

06 / K-means

Mechanism: Alternates nearest-centroid assignment and centroid updates to reduce within-cluster squared distances. **Example:** Group customers using purchase frequency and spending.
Selection and limitation: Choose K, scale features when appropriate, and inspect cluster stability. Groups are not automatically meaningful labels. [3]

07 / PCA

Mechanism: Projects centered data onto orthogonal directions that retain maximum variance for a chosen dimension. **Example:** Compress correlated network traffic measurements.
Selection and limitation: High variance need not mean predictive importance. Feature scaling changes results; inspect explained variance and downstream utility. [4]

08 / GPT-style transformer

Mechanism: Uses attention over preceding tokens and learns next-token prediction; generation repeats that prediction. **Example:** Draft a support response for human review.
Selection and limitation: Fluent output may be inaccurate. Evaluate factual correctness and task relevance with a representative review set. [6]

09 / GAN

Mechanism: Trains a generator against a discriminator that distinguishes real from generated samples. **Example:** Produce synthetic product images for a controlled experiment.
Selection and limitation: Adversarial training can be unstable or lose diversity through mode collapse. Check fidelity and diversity; synthetic data does not guarantee better prediction. [7]

10 / Diffusion model

Mechanism: Learns a reverse denoising process from deliberately corrupted examples; sampling progressively transforms noise into data. **Example:** Generate candidate concept images.
Selection and limitation: Iterative generation can be computationally costly. Inspect fidelity, diversity, and suitability; conditioning depends on the model setup. [8]

Four areas, overlapping uses

The assignment uses four application areas. Generative AI is included explicitly below as an application area, while remaining a task-oriented category that overlaps with NLP and computer vision. Primary = direct fit in this guide; Features = use after numeric feature extraction; - = not a primary use shown.

Algorithm / family	Tabular data	Computer vision	NLP	Generative AI
Linear regression	Primary	Features	Features	-
Decision tree	Primary	Features	Features	-
Random forest	Primary	Features	Features	-
Gradient boosting	Primary	Features	Features	-
CNN classifier	-	Primary	-	-
K-means	Primary	Features	Features	-
PCA	Primary	Features	Features	-
GPT-style transformer	-	-	Primary	Primary: text
GAN	-	Primary: images	-	Primary: images
Diffusion model	-	Primary: images	Text conditioning	Primary: images

Generative AI: same goal, different mechanisms

GPT: extends a token sequence, useful for drafting and summarization. **GAN:** uses generator-discriminator competition, useful for synthetic-image experiments. **Diffusion:** learns denoising, useful for image creation and editing. Stable Diffusion is a text-conditioned latent diffusion example: it denoises a compressed representation and decodes it into an image [6]-[9].

A text-conditioned image system spans NLP, vision, and generative AI. Its text-image pairs provide conditioning information, so its full training pipeline should not be described simply as unlabeled learning. The unconditional denoising setup in the classification table is a simpler case [8], [9].

NLP understanding versus generation

GPT is the generative NLP example among the ten entries. BERT is a useful comparison: its bidirectional masked-token pretraining learns language representations, and supervised fine-tuning can support sentiment classification. BERT and GPT are both transformers, but a sentiment label and a newly generated passage are different task outputs [6], [10].

How the methods connect

Shared family / goal	Relationship	Key distinction
Tree predictors	Decision tree → random forest / gradient boosting	Forest: independent randomized trees, then aggregate. Boosting: sequential loss-reducing additions. [1], [2]
Unlabeled feature analysis	PCA can precede K-means in a pipeline	PCA compresses features; K-means groups examples. Neither creates external class labels. [3], [4]
Generative modeling	GPT / GAN / diffusion share a generation goal	GPT: autoregressive tokens. GAN: adversarial competition. Diffusion: learned iterative denoising. [6]-[8]
Vision applications	CNN classifier / image GAN / image diffusion	Same domain, different outputs: class labels versus new images. [5], [7], [8]

A practical selection path

Known target? For a number, start with regression; for a category, compare classifiers. On tabular data, establish a simple baseline before comparing forests or boosting. For labeled images, consider a CNN. **No task labels?** Use K-means to explore groups or PCA to reduce dimensions. **Need new content?** Compare generative models appropriate to the output modality [1]-[8].

Learning reflection - draft for personal review

This comparison helps me distinguish how a model learns from what it produces. The most challenging boundary to explain is that generative models can have training targets without needing externally supplied class labels. Next-token prediction and denoising show why the source of the target matters more than simply asking whether a target exists.

Organizing the guide around three questions gives me a strategy for checking ambiguous cases: name the training setup, identify its targets, and then describe its input and output. It also helps me see why K-means and PCA belong together under unsupervised learning while solving different problems. A cluster groups observations; a principal component describes a direction of variation.

For my interests in AI and networking, this distinction connects course concepts to possible applications. Labeled traffic events suggest supervised classification; unlabeled traffic patterns suggest exploratory clustering. My next step is to compare a simple predictor with an ensemble using a fixed data split and suitable metrics. I would fit preprocessing only on training data and judge performance on held-out examples. This guide is conceptual; it does not report a trained model or measured results.

Sources and documentation

Numeric citation style is used throughout. References are numbered by first appearance. All authors are listed for the research papers. Documentation accessed November 14, 2026.

[1] scikit-learn developers, "Supervised learning," scikit-learn documentation.
https://scikit-learn.org/stable/supervised_learning.html

[2] scikit-learn developers, "Ensembles: Gradient boosting, random forests, bagging, voting, stacking," scikit-learn documentation.
https://scikit-learn.org/stable/modules/ensemble.html

[3] scikit-learn developers, "Clustering," scikit-learn documentation.
https://scikit-learn.org/stable/modules/clustering.html

[4] scikit-learn developers, "Decomposing signals in components (matrix factorization problems)," scikit-learn documentation.
https://scikit-learn.org/stable/modules/decomposition.html

[5] TensorFlow, "Convolutional Neural Network (CNN)," TensorFlow Core tutorials.
https://www.tensorflow.org/tutorials/images/cnn

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," arXiv:2005.14165, 2020.
https://arxiv.org/abs/2005.14165

[7] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," arXiv:1406.2661, 2014.
https://arxiv.org/abs/1406.2661

[8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," arXiv:2006.11239, 2020.
https://arxiv.org/abs/2006.11239

[9] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," CVPR, 2022. Stable Diffusion implementation: CompVis/stable-diffusion.
https://github.com/CompVis/stable-diffusion

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv:1810.04805, 2018.
https://arxiv.org/abs/1810.04805

Use cases are illustrative. Domain assignments describe the setups shown rather than exclusive capabilities. The learning reflection is a proposed draft and should be personalized before academic submission.